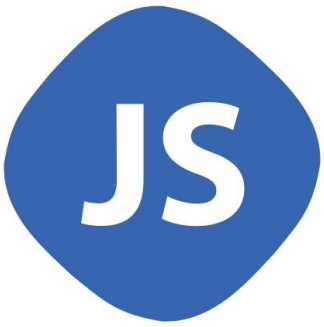


Массивы

- Общая информация о массивах
- Одномерные массивы
- Многомерные массивы
- Стек и очередь
- Методы для добавления и удаления элементов
- Методы в динамическом создании массивов

Общая информация о массивах

В языках программирования информация представлена разными **типами данных**: целые числа, числа с плавающей точкой, строки, логические значения и другие. Благодаря типам компьютер понимает, как хранить ту или иную информацию и какие операции с ней можно выполнять.

Но есть более сложные сущности - **структуры данных**, которые, как правило, содержат данные разных типов, связанные между собой определенным образом. Одна из таких структур - массивы.

> ***Примечание***. Мы много работали с массивами, в этом уроке рассмотрим логику формирования и способы применения массивов.

Массив (Array) – это специальная структура данных, которая предназначена для хранения значений **упорядоченных коллекций**.

Массивы очень полезны и очень часто используются в коде, т.к. позволяют хранить набор значений в одной переменной.

Например, вместо того чтобы объявлять несколько переменных для хранения выученных языков программирования, можно объявить одну со всеми этими значениями:

```
// набор переменных простых типов
let js = "javaScript";
let php = "php";
let pl = "perl";
let py = "python";
// массив элементов простых типов
let lang = ["javaScript", "php", "perl", "python"];
```

Особенно актуален такой подход, если вы не планируете останавливаться на изучении языков программирования и технологий.

Значения массива называются элементами. Каждый элемент в массиве имеет свой **порядковый номер** (**индекс**). Первый элемент массива имеет индекс 0, второй – 1, третий – 2 и т.д.



Но по настоящему достоинства массивов проявляются при необходимости обрабатывать и выводить **большие объемы структурированных данных**.

Объявление массива

Создание пустого массива в JavaScript можно выполнить двумя способами, с помощью:

- **литерала** массива;
- **конструктора Array**().

```
// использование литерала массива
let arr = [];
// использование конструктора Array()
let arr = new Array();
```

Метод создания массива с помощью литерала (квадратных скобок) **более предпочтителен** и в большинстве случаев лучше использовать именно его.

При объявлении массива его можно сразу **инициализировать начальными значениями**. Для этого внутрь скобок необходимо поместить элементы, отделив их друг от друга запятой:

```
// инициализация массива при объявлении с помощью литерала
let lang = ["javaScript", "php", "perl", "python"];
// инициализация массива при объявлении с помощью конструктора
let lang = new Array("javaScript", "php", "perl", "python");
```

> ***Внимание***. Если конструктору **Array**() передать один аргумент, который является числом, то он создаст массив с указанным **количеством элементов**. Значения элементов при этом будут **неопределёнными** (пустыми)

Например:

```
let arr = new Array(3);
console.log(arr) // [ , , ]
```

example_1. Объявление массива

```
<script>
    // объявляем массив
    // инициализируем массив начальными значениями
    // литерал
```

```
    var film = ["Зеленая миля", "Криминальное чтиво", "Форрест
    Гамп"];
    // конструктор
    let books = new Array("Чемодан", "Зона", "Соло на IBM")
    // выводим данные массивов
    console.log(film);
    console.log(books);
</script>
```

Каким бы способом ни создавался массив, впоследствии, с его элементами всегда можно продолжить работать: **добавить**, **обновить**, **удалить**.

Доступ к элементам массива

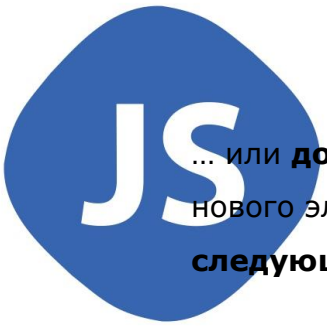
Получение доступа к элементу массива выполняется через квадратные скобки, внутрь которых нужно поместить индекс. **Индексы массива нумеруются, начиная с нуля**.

Так как индексы нумеруются с 0, то для получения первого, второго и третьего элементов нужно использовать индексы 0, 1 и 2.

```
let lang = ["Зеленая миля","Криминальное чтиво","Форрест Гамп"];
alert( lang[0] ); // Зеленая миля
alert( lang[1] ); // Криминальное чтиво
alert( lang[2] ); // Форрест Гамп
```

Элемент массива можно **обновить**:

```
lang[2] = 'Общество мертвых поэтов';
alert( lang[2] ); // Общество мертвых поэтов
```



… или **добавить** новый элемент к существующему массиву. Для добавления нового элемента в массив, можно просто присвоить нужное значение **следующему индексу**.

example_2. Обновление элементов массива

```
<script>
    // объявляем массив
    // инициализируем массив начальными значениями
    var film = ["Зеленая миля", "Криминальное чтиво", "Форрест Гамп"];
    // обновляем элемент массива
    film[2] = "Общество мертвых поэтов";
    // добавляем элементы в массив
    film[3] = "Бойцовский клуб";
    film[4] = "Умница Уилл Хантинг";
    film[5] = "Побег из Шоушенка";
    // выводим обновленные данные массива
    console.log(film);
</script>
```

Если при добавлении элемента случайно пропустить индекс, то в массиве появится неопределенный элемент **undefined**:

```
let lang = ["javaScript","php","perl","python"];
lang[5] = "visual basic";
console.log( lang[4] ); // undefined
alert(lang); // вывести массив целиком можно при помощи alert
```

При попытке получить доступ к несуществующему элементу также получим **undefined**:

```
let lang = new Array ("Doors","Aerosmith","Creedence");
console.log( lang[3] ); // undefined
```

Совсем необязательно, чтобы все элементы массива имели один и тот же тип данных. Элементами могут быть **любые значения** – даже другие массивы. Это позволяет создавать **сложные структуры данных**, например, массивы массивов или массивы объектов (об этом далее).

В приведенном ниже примере **не нужно** погружаться в изучение объектов или функций, необходимо уяснить – **элементами массива может быть все что угодно**.

example_3. В массиве могут храниться элементы любого типа

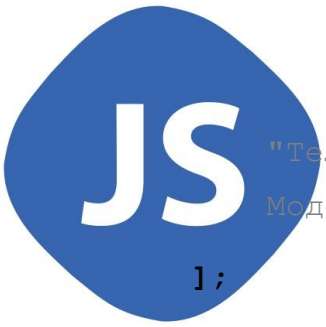
```
<script>
    // массив может хранить значения разного типа
    // 0 - строка
    // 1 - объект
    // 2 - boolean
    // 3 - функция
    // 4 - другой массив
    let arr = [ 'Яблоко', {name: 'Джон'}, true, function()
    {return('Привет');}, ['PHP', 'JavaScript'] ];
    // получить элемент с индексом 1 (объект),
    // затем показать его свойство
    alert( arr[1].name ); // Джон
    // получить элемент с индексом 3 (функция) и выполнить её
    alert(arr[3]()); // Привет
    // получить элемент с индексом 4 (массив)
    // затем вывести его второй элемент
    alert(arr[4][1]); // JavaScript
```

```
</script>
```

Мы рассмотрели несколько массивов, но приведенные примеры [*не совсем*/*совсем не*] отражают реальные массивы, применяемые в веб-разработке. Чаще всего мы будем составлять и работать с массивами, представляющими некоторые **объекты реального мира** определенной **предметной области**.

Например, ниже представлены два массива, содержащие данные (объекты реального мира) из разных предметных областей.

```
// элементы массива – это свойства альбома группы Pink Floyd
let album = [
    1,
    "Atom Heart Mother",
    "10 октября 1970",
    "EMI, Harvest, Capitol",
    "LP, CD",
    "Золотой (USA)"
];
// элементы массива – это свойства телевизора бренда Hyundai
let television = [
    "55",
    "LED",
    "60 Гц",
    "Hyundai",
    "H-LED55GU7000",
    "4K UltraHD",
    40000,
    "img/H-LED55GU7000.jpg",
```



```
    "Телевизор Hyundai оснащен экраном с диагональю 140 см.
    Модель Hyundai H-LED55GU7000 обладает ..."
];
```

Различают **одномерные** и **двумерные** массивы. Одномерный массив – это **список переменных**, двумерный массив – **таблица**, имеющая строки и столбцы.

Элементы одномерного массива обозначаются одним индексом, он определяет порядковый номер элемента в массиве (как элемент имеет свой номер в списке). Элементы двумерного массива обозначаются двумя индексами (как ячейка обозначается номером строки и столбца).

Измерение массива определяется количеством индексов, необходимых для выбора элемента. Для адресации элемента двумерного массива, необходимо два индекса, трехмерного – три, и т.д.

> ***Важно***. **Размерность массива** — это количество индексов, необходимое для **однозначной адресации** элемента в рамках массива.

Подробнее о логике составления одномерных и двумерных массивах далее.

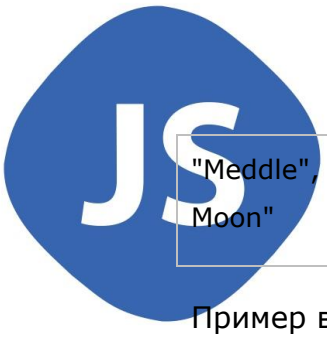
Одномерные массивы

Одномерный массив — структура данных, хранящая набор значений (элементов массива), идентифицируемых по индексу, принимающему значения из некоторого заданного непрерывного диапазона.

Одномерный массив можно рассматривать как реализацию абстрактного типа данных — **вектор**.

> ***К сведению***. **Вектор** - одномерный массив проиндексированных элементов.

Попробуем визуализировать одномерный массив в виде **векторного представления**.



"Meddle", "Obscured by Clouds", "The Dark Side of the Moon", "The Dark Side of the Moon"

Пример вектора в объявлении одномерного массива:

```
let albums = ["Meddle", "Obscured by Clouds", " The Dark Side of the Moon"];
```

Это в самом простом случае. Но массив — это структура данных, позволяющая хранить гораздо более информативные, а поэтому и более сложные значения.

В случае с альбомами, набор значений для каждого элемента массива может выглядеть, например, следующим образом:

```
// одномерный массив данных об альбомах группы Pink Floyd
let albums = [
    "2; Meddle; 30 октября 1971; EMI, Harvest, Capitol; Vinyl, Кассета, CD; Платиновый (USA)",
    "3; Obscured by Clouds; 3 июня 1972; EMI, Harvest, Capitol; LP, Кассета, CD; Золотой (USA), Серебряный (GBR)",
    "4; The Dark Side of the Moon; 17 марта 1973; Harvest, Capitol, EMI; LP, Кассета, CD, SACD; Платиновый (USA), Платиновый (GBR), Бриллиантовый (CAN)"
];
```

Каждое значение такого элемента массива описывает **набор свойств** определенного альбома. И пусть такое отображение несколько условно, но теперь получим следующее **векторное представление** одномерного массива:

"2; Meddle; 30 октября 1971; EMI, Harvest, Capitol; Vinyl, Кассета, CD; Платиновый (USA)", "3; Obscured by Clouds; 3 июня 1972; EMI, Harvest, Capitol; LP, Кассета, CD; Золотой (USA), Серебряный (GBR)", "4; The Dark Side of the Moon; 17 марта 1973; Harvest, Capitol, EMI; LP, Кассета, CD, SACD; Платиновый (USA), Платиновый (GBR), Бриллиантовый (CAN)"

Нужно ли говорить, что такое представление не является оптимальной структурой хранения данных, даже при небольшом количестве составляющих массив элементов (элементов всего три).

Руководствуясь жизненным опытом представления данных, мы знаем, что для структурирования большого количества информации используют таблицы. Если речь идет о программировании, то это **таблицы баз данных**.

> ***Примечание***. Курс нацелен на создание клиент-серверных приложений. Фронтенд разработчик обязан знать терминологию и основы проектирования баз данных (но это уже совсем другая история). Некоторые важные моменты рассмотрим в разделе **Многомерные массивы**.

Многомерные массивы

Двумерный массив

Таблица — это структура, хранящая информацию о **совокупности объектов**, которые имеют схожие характеристики.

> ***Примечание***. Объекты, хранимые в таблицах баз данных часто называют общим понятием - **объекты реального мира**.

Таблица состоит из **одной** или **множества строк**, каждая строка содержит информацию об одном объекте реального мира.

Преобразуем **векторное представление** в **двумерную таблицу**.

"2; Meddle; 30 октября 1971; EMI, Harvest, Capitol; Vinyl, Кассета, CD; Платиновый (USA)"
"3; Obscured by Clouds; 3 июня 1972; EMI, Harvest, Capitol; LP, Кассета, CD; Золотой (USA), Серебряный (GBR)"



"4; The Dark Side of the Moon; 17 марта 1973; Harvest, Capitol, EMI; LP, Кассета, CD, SACD; Платиновый (USA), Платиновый (GBR), Бриллиантовый (CAN)"

В приведенной двумерной таблице каждая строка представляет набор данных об одном альбоме. Представить получившуюся таблицу можно с помощью переменной **двумерного массива**:

```
// объявление двумерного массива
let albums = [
    ["2; Meddle; 30 октября 1971; EMI, Harvest, Capitol; Vinyl, Кассета, CD; Платиновый (USA)"],
    ["3; Obscured by Clouds; 3 июня 1972; EMI, Harvest, Capitol; LP, Кассета, CD; Золотой (USA), Серебряный (GBR)"],
    ["4; The Dark Side of the Moon; 17 марта 1973; Harvest, Capitol, EMI; LP, Кассета, CD, SACD; Платиновый (USA), Платиновый (GBR), Бриллиантовый (CAN)"]
];
```

Двумерный массив — это одномерный массив (**внешний** массив), элементами которого являются одномерные массивы (**вложенные** массивы). Другими словами, это набор структурированных данных, имеющих общее имя, в которых доступ к элементам осуществляется по двум индексам.

Первый индекс идентифицирует номер **вложенного массива**, второй индекс - **номер элемента** текущего массива.

В демонстрационном примере **example_4** протестируем доступ к объектам альбома в массиве.

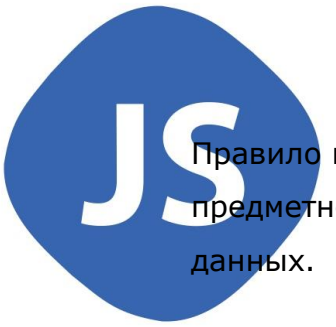
example_4. Двумерный массив неатомарных значений

```
<script>
    let albums = [
        ["2; Meddle; 30 октября 1971; EMI, Harvest, Capitol;
```

```
            Vinyl, Кассета, CD; Платиновый (USA)"],
            ["3; Obscured by Clouds; 3 июня 1972; EMI, Harvest,
            Capitol; LP, Кассета, CD; Золотой (USA), Серебряный
            (GBR)"],
            ["4; The Dark Side of the Moon; 17 марта 1973; Harvest,
            Capitol, EMI; LP, Кассета, CD, SACD; Платиновый (USA),
            Платиновый (GBR), Бриллиантовый (CAN)"]
        ];
        // исходный массив albums
        console.log("Исходный массив: ");
        console.log(albums);
        // второй вложенный массив
        console.log("Второй вложенный массив: ");
        console.log(albums[1]);
        // первый элемент второго вложенного массива
        console.log("Первый элемент второго вложенного массива: ");
        console.log(albums[1][0]);  // пример неатомарных данных
</script>
```

В приведенном примере каждый элемент массива представляет из себя **текстовый набор разнотипных значений** и не является **атомарным**. Это значит, что в **наборе текстовых символов** затруднительно получить доступ, например, к названию, дате выхода или статусу альбома.

Важно. **Атомарность данных** - важное понятие из теории разработки реляционных баз данных. Данные ячеек таблицы должны быть семантически неделимы. Это значит, что в проектируемой базе данных не должно быть столбца **ФИО**, должны быть столбцы: **Фамилия**, **Имя**, **Отчество**. Тоже самое относительно столбцов: **Адрес**, **Паспортные Данные** и некоторых других.



Правило важное и о нем надо знать, но справедливости ради, многое зависит от предметной области и требований к нормализации проектируемой базы данных.

Создавая скрипт для обработки и последующего вывода некоторой информации, мы должны предусмотреть **легко расширяемую структуру**, имеющую простой алгоритм обработки больших объемов данных.

Атомизация данных упрощает обработку данных, но приводит к усложнению структуры. Как правило, это приводит к увеличению размерности массива.

> ***Важно***. **Многомерный массив** - массив, содержащий один или несколько вложенных массивов.

Теория реляционных баз данных предполагает, что в таблицах хранятся данные об объектах реального мира. Каждый объект **представляется определенным набором свойств** (атрибутов). Каждое свойство должно быть **атомарно**, и иметь значение **определенного типа**.

Тогда данные об альбомах, трансформируются из **строкового** литерала:

```
["2; Meddle; 30 октября 1971; EMI, Harvest, Capitol; Vinyl, Кассета, CD; Платиновый (USA)"],
```

... к **атомарному** представлению:

```
[2, "Meddle", "30 октября 1971", "EMI, Harvest, Capitol", "Vinyl, кассета, CD", "Платиновый (USA)"];
```

Заменим каждое вхождение альбома во вложенном массиве атомарным представлением.

Массив **albums**:

```
let albums = [
    [2, "Meddle", "30 октября 1971", "EMI, Harvest, Capitol", "Vinyl, Кассета, CD", "Платиновый (USA)"],
```

```
        [3, "Obscured by Clouds", "3 июня 1972", "EMI, Harvest,
        Capitol", "LP, Кассета, CD", "Золотой (USA), Серебряный
        (GBR)"],
        [4, "The Dark Side of the Moon", "17 марта 1973", "Harvest,
        Capitol, EMI", "LP, Кассета, CD, SACD", "Платиновый (USA),
        Платиновый (GBR), Бриллиантовый (CAN)"]
];
```

Таким образом, получим двумерный массив **albums**, который можно представить в виде следующей **таблицы**:

2	Meddle	30 октября 1971	EMI, Harvest, Capitol	Vinyl, Кассета, CD	Платиновый (USA)
3	Obscured by Clouds	3 июня 1972	EMI, Harvest, Capitol	LP, Кассета, CD	Золотой (USA), Серебряный (GBR)
4	The Dark Side of the Moon	17 марта 1973	Harvest, Capitol, EMI	LP, Кассета, CD, SACD	Платиновый (USA), Платиновый (GBR), Бриллиантовый (CAN)

В примере **example_5** реализовано программное объявление и доступ к **переменной двумерного массива albums**.

example_5. Двумерный массив атомарных значений

```
<script>
    let albums = [
        [2, "Meddle", "30 октября 1971", "EMI, Harvest, Capitol",
        "Vinyl, Кассета, CD", "Платиновый (USA)"],
        [3, "Obscured by Clouds", "3 июня 1972", "EMI, Harvest,
        Capitol", "LP, Кассета, CD", "Золотой (USA), Серебряный
        (GBR)"],
        [4, "The Dark Side of the Moon", "17 марта 1973",
        "Harvest, Capitol, EMI", "LP, Кассета, CD, SACD",
        "Платиновый (USA), Платиновый (GBR), Бриллиантовый
        (CAN)"]
```

```
    ];
    console.log("Название альбома: ");
    console.log(albums[1][1]); // пример атомарных данных
</script>
```

Вполне закономерен вопрос, что за данные находятся в каждом из столбцов представленного массива? Эту информацию придется запоминать, ну или **писать комментарий** в сценарии программы, например, такой:

```
/*
    Индекс - Расшифровка
    0 - Код
    1 - Название
    2 - Дата выхода
    3 - Лейбл
    4 - Формат
    5 - Статус
*/
```

Свести процесс запоминания к минимуму, можно используя вместо вложенных массивов **объекты**.

Иногда массивы объектов – лучшее решение

По мере того, как вы будете заполнять массив, код может стать всё менее поддерживаемым. Один из способов **структурирования данных** – использовать вложенные объекты.

example_6. Пример массива объектов

```
<script>
    let albums = [
```

```
        {
            id: 2,
            name: "Meddle",
            date: "30 октября 1971",
            label: "EMI, Harvest, Capitol",
            format: "Vinyl, Кассета, CD",
            status: "Платиновый (USA)"
        },
        { // ... },
        { // ... }
    ];
    with(albums[1]) {
        document.write(`
        <pre>
            id: ${id},
            Название: ${name},
            Дата выхода: ${date},
            Лейбл: ${label},
            Формат: ${format},
            Статус: ${status}
        </pre>
        `);
    }
</script>
```

Примечание. **Массивы** нужно использовать тогда, когда имеет значение

последовательность информации.

Объекты — лучший выбор, если вы хотите структурировать информацию по названию данных.

Вообще, в массивах и объектах можно хранить разные структуры данных. Для лучшей **масштабируемости кода** вам практически всегда нужна будет комбинация из нескольких структур:

- массивы внутри объектов;
- объекты внутри массивов;
- массивы внутри массивов;
- объекты внутри объектов.

Как обычно, все зависит от поставленных программисту задач и предметной области. Но, выбирая структуру данных, помните, **объекты внутри внешнего массива** – классическая структура реляционных баз данных.

Как получаются многомерные массивы

Еще раз обратим внимание на **атомарность данных,** хранящихся в массиве.

Допустим, в своем распоряжении мы имеем двумерный массив, представленный следующей таблицей.

id	name	date	label	format	status
2	Meddle	30 октября 1971	EMI, Harvest, Capitol	Vinyl, Кассета, CD	Платиновый (USA)
3	Obscured by Clouds	3 июня 1972	EMI, Harvest, Capitol	LP, Кассета, CD	Золотой (USA), Серебряный (GBR)
4	The Dark Side of the Moon	17 марта 1973	Harvest, Capitol, EMI	LP, Кассета, CD, SACD	Платиновый (USA), Платиновый (GBR), Бриллиантовый (CAN)
5	Wish You Were Here	15 сентября 1975	Harvest, EMI, Columbia, Capitol	LP, 8-track, Кассета, CD, SACD	Платиновый (USA), Золотой (GBR), Платиновый (CAN)

Простой анализ данных показывает, что столбцы **id**, **name**, **date** – имеют **простой** (скалярный) тип данных. Значения в этих столбцах атомарны.

Тогда как столбцы **label**, **format**, **status** содержат **перечисляемые значения**. Данные этих столбцов не атомарны. Это значит, что получить доступ к статусу альбома "The Dark Side of the Moon" в Великобритании не так-то просто, придется разбирать строковое значение.

> ***К сведению***. Доступ получить можно, но постоянный **парсинг** (обработка) **строковых элементов** массива будет занимать дополнительные аппаратные и временные ресурсы.

Выходом является дальнейшее усложнение структуры массива.

Во избежание подобных ситуаций такие значения принято представлять в виде массива, теперь уже массива третьего уровня. Заменим значения **строкового литерала** неатомарных столбцов **вложенным массивом третьего уровня**.

Для **трехмерного массива**, необходимо **три индекса** для выбора элемента.

Пример массива **третьего уровня** и способ доступа к его элементам представлен в демонстрационном коде **example_7**.

example_7. Трехмерный массив и доступ к его элементам

```
<script>
    let albums = [
        [
            2,
            "Meddle",
            "30 октября 1971",
            ["EMI", "Harvest", "Capitol"],
            ["Vinyl", "Кассета", "CD"],
            ["Платиновый (USA)"]
```

```
    ],
    [
        3,
        "Obscured by Clouds",
        "3 июня 1972",
        ["EMI", "Harvest", "Capitol"],
        ["LP", "Кассета", "CD"],
        ["Золотой (USA)", "Серебряный (GBR)"]
    ],
    [
        4,
        "The Dark Side of the Moon",
        "17 марта 1973",
        ["Harvest", "Capitol", "EMI"],
        ["LP", "Кассета", "CD", "SACD"],
        ["Платиновый (USA)", "Платиновый (GBR)",
        "Бриллиантовый (CAN)"]
    ]
];
document.write(`
    <pre>
        id: ${albums[2][0]},
        Название: ${albums[2][1]},
        Дата выхода: ${albums[2][2]},
        Лейбл: ${albums[2][3][0]}, ${albums[2][3][1]},
        ${albums[2][3][2]}
        Формат: ${albums[2][4][0]}, ${albums[2][4][1]},
```

```
            ${albums[2][4][2]}, ${albums[2][4][3]}
            Статус: ${albums[2][5][0]}, ${albums[2][5][1]},
            ${albums[2][5][2]}
        </pre>
    `);
</script>
```

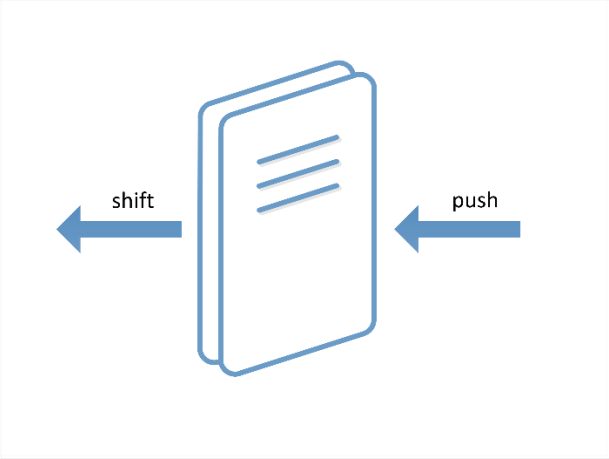
Статус альбома "Meddle" имеет строковое скалярное значение, тем не мене он так же должен быть в массиве. Структура данных объектов должна быть одинакова.

Стек и очередь

Две из наиболее часто используемых структур данных в веб-разработке — это стеки и очереди.

Очередь – один из самых распространённых вариантов применения массива. В области компьютерных наук так называется упорядоченная коллекция элементов, поддерживающая два вида операций:

- **push** – добавляет элемент в конец.
- **shift** – удаляет элемент в начале, сдвигая очередь, так что второй элемент становится первым.

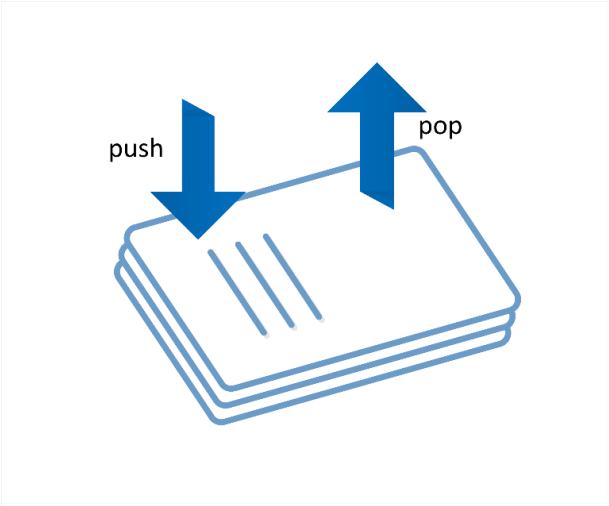


На практике необходимость в этом возникает очень часто. Например, очередь сообщений, которые надо показать на экране.

Другой вариант применения для массивов – структура данных, называемая **стек**.

Стек поддерживает два вида операций:

- **push** – добавляет элемент в конец.
- **pop** – удаляет последний элемент.

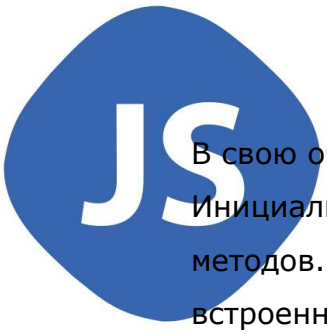


Таким образом, новые элементы всегда добавляются или удаляются из **конца**.

Массивы в JavaScript могут работать и как **очередь**, и как **стек**. С помощью специальных методов можно добавлять/удалять элементы как в начало, так и в конец массива.

Методы для добавления и удаления элементов

Стандарт ECMAScript определяет такой тип данных как Объект (Object). Объект JavaScript может расцениваться как **структура**, содержащая набор свойств и методов.



В свою очередь массивы в JavaScript — это **особые виды объектов**. Инициализация массива задаёт определённое количество начальных свойств и методов. Реальная сила массивов JavaScript заключается именно в их встроенных свойствах и методах.

Доступ и обращение к **свойствам** и **методам** массива осуществляется при помощи **точечной нотации** (через символ "**.**"):

Array.свойство

Array.метод (параметры)

Массивы в JavaScript имеют специальные **методы для добавления элементов**:

- **push**() – для добавления **одного** или **нескольких** элементов в конец массива;
- **unshift**() – для добавления **одного** или **нескольких** элементов в начало массива.

Методы удаления элементов:

- **pop**() – для удаления **последнего** элемента;
- **shift**() – для удаления **первого** элемента.

Методы, работающие с концом массива

push

Добавляет элемент в конец массива:

```
let langs = ["javaScript", "php", "perl"];

langs.push("python");

alert(langs); // javaScript, php, perl, python
```

Метод **push** позволяет добавлять сразу несколько элементов.

```
langs.push("python", "vueJS");
```



pop

Удаляет последний элемент из массива и возвращает его:

```
let langs = ["javaScript", "php", "perl", "python"];
alert(langs.pop()); // удаляем python и выводим его
alert(langs); // javaScript, php, perl
```

example_8. Методы, работающие с концом массива

```
<script>
    // инициировали массив альбомов группы Pink Floyd
    let albums = ["Atom Heart Mother", "Meddle"];
    // добавим в конец массива новый элемент
    albums.push("Obscured by Clouds");
    // добавим два элемента
    albums.push("The Dark Side of the Moon", "Animals");
    // последний элемент удалим и сохраним в переменной
    let el = albums.pop();
    // выведем результат
    console.log("Массив: ", albums);
    console.log("Удаленный элемент: ", el);
</script>
```

Пример использования методов для многомерного массива.

example_9. Методы push и pop в многомерных массивах

```
<script>
    // инициировали массив альбомов группы Pink Floyd
    var albums = [ [ // ... ], [ // ... ], [ // ...  ] ];
    var newAlbum =  [
```

```
    [
        6,
        "Animals",
        "23 января 1977",
        "Harvest, EMI, Columbia, Capitol",
        "LP, 8-track, кассета, CD",
        "Платиновый (USA), Золотой (GBR), Платиновый (CAN)"
    ],
    [
        7,
        "The Wall",
        "30 ноября 1979",
        "Harvest, EMI, Columbia, Capitol",
        "LP, 8-track, кассета, CD",
        "Платиновый (USA), Платиновый (GBR), Бриллиантовый
        (CAN)"
    ]
];
// выведем исходный массив
console.log("Исходный массив: ", albums);
// добавим в конец массива новый элемент
albums.push([
5,
"Wish You Were",
"15 сентября 1975",
"Harvest, EMI, Columbia, Capitol",
```

```
    "LP, 8-track, кассета, CD, SACD",
    "Платиновый (USA), Золотой (GBR), Платиновый (CAN)"
    ]);
    // добавили в массив элемент
    console.log("Исходный массив + новый элемент: ", albums);
    // добавим два элемента
    albums.push(newAlbum[0], newAlbum[1]);
    // выведем обновленный массив
    console.log("Обновленный массив: ", albums);
    // последний элемент удалим и сохраним в переменной
    let el = albums.pop();
    // выведем результирующий массив
    console.log("Результирующий массив: ", albums);
    console.log("Этот массив был удален: ", el);
</script>
```

Методы, работающие с началом массива

unshift

Добавляет элемент в начало массива.

```
let langs = ["php","perl","python"];
langs.unshift("javaScript");
alert(langs); // javaScript, php, perl, python
```

Метод **unshift** позволяет добавлять сразу несколько элементов.

```
langs.unshift("javaScript", "css");
```

shift

Удаляет из массива первый элемент и возвращает его.

```
let langs = ["javaScript","php","perl","python"];
alert(langs.shift()); // удаляем javaScript и выводим его
alert(langs); // php, perl, python
```

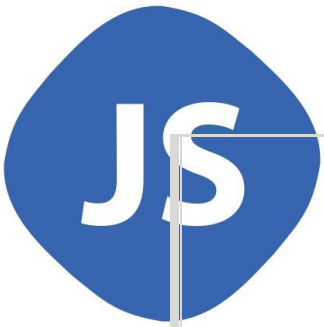
example_10. Методы, работающие с началом массива

```
<script>
    // инициируем массив
    let albums = ["Меркурий", "Венера"];
    // добавим в начало массива новый элемент
    albums.unshift("Земля");
    // добавим в начало еще три элемента
    albums.unshift("Плутон", "Юпитер", "Сатурн");
    // первый элемент удалим и сохраним в переменной
    let el = albums.shift();
    // выведем результат
    console.log("Массив: ", albums);
    console.log("Удаленный элемент: ", el);
</script>
```

Пример использования методов для многомерного массива.

example_11. Методы unshift и shift в многомерных массивах

```
<script>
    // инициировали массив альбомов группы Pink Floyd
    var albums = [ [ // ... ], [ // ... ], [ // ...  ] ]
    var newAlbum = [
        6,
```



```
        "Animals",
        "23 января 1977",
        "Harvest, EMI, Columbia, Capitol",
        "LP, 8-track, кассета, CD",
        "Платиновый (USA), Золотой (GBR), Платиновый (CAN)"
];
// выведем исходный массив
console.log("Исходный массив: ", albums);
// добавим в начало массива новый элемент
albums.unshift([
    1,
    "Atom Heart Mother",
    "10 октября 1970",
    "EMI, Harvest, Capitol",
    "LP, CD",
    "Золотой (USA)"
]);
// добавили в массив элемент
console.log("Исходный массив + новый элемент: ", albums);
// добавим еще один элемент
albums.unshift(newAlbum);
// удалим первый элемент и сохраним его в переменной
let el = albums.shift();
// выведем результирующий массив
console.log("Результирующий массив: ", albums);
console.log("Удаленный массив: ", el);
```



```
</script>
```

Методы в динамическом создании массивов

Как обычно, не обойдемся без циклических конструкций. Протестируем работу методов по добавлению данных в массивы в циклах.

Запись в одномерный массив

Сохраним данные абзацев документа в простой **одномерный** массив.

example_12. Сохранение в одномерный массив

```
<script>
    // получение коллекции элементов абзацев
    let elements = document.querySelectorAll('div#citate p');
    // количество элементов коллекции
    let len = elements.length;
    // инициируем переменные
    let arr = [];
    var i = 0;
    // цикл while
    while (i < len) {
        // заполняем массив
        arr.push(elements.item(i).innerText);
        i++;
    }
    // выводим массив в консоль
```

```
    console.log(arr);
    // доступ к одномерному массиву через один индекс
    console.log(arr[2]);
</script>
```

Запись в многомерный массив

Усложним задачу, сохраним табличные данные в **двумерный** массив. Используем вложенную конструкцию цикла.

example_13. Сохранение в двумерный массив

```
<script>
    // получение коллекции элементов списка
    let tr = document.querySelectorAll('table > tbody > tr');
    // количество элементов коллекции
    let len = tr.length;
    // инициируем массив
    let franchise = [];
    // внешний цикл for
    for (let i=0; i < len; i++) {
        franchise[i] = [];
        td = tr.item(i).querySelectorAll("td");
        franchise[i] = [];
        // вложенный цикл for
        for  (k=0; k < td.length; k++) {
            // заполняем массив третьего уровня
            franchise[i].push(td.item(k).innerText);
        }
```

```
    }
    // выводим массив в консоль
    console.log(franchise);
    // доступ к двумерному массиву через два индекса
    console.log(franchise[1][0]);
</script>
```

И еще раз усложним задачу. Сохраним эти же табличные данные теперь в **трехмерный** массив.

example_14. Сохранение в трехмерный массив

```
<script>
    // получение коллекции элементов списка
    let tr = document.querySelectorAll('table > tbody > tr');
    // console.log(tr);
    // количество элементов коллекции
    let len = tr.length;
    // инициируем массив
    let franchise = [];
    // внешний цикл for
    for (let i=0; i < len; i++) {
        franchise[i] = [];
        td = tr.item(i).querySelectorAll("td");
        // console.log(td);
        franchise[i].push(td.item(0).innerText);
        franchise[i][1] = [];
        // вложенный цикл for
```

```
        for  (k=1; k < td.length; k++) {
            // заполняем массив третьего уровня
            franchise[i][1].push(td.item(k).innerText);
        }
    }
    // выводим массив в консоль
    console.log(franchise);
    // доступ к трехмерному массиву через три индекса
    console.log(franchise[1][1][4]);
</script>
```

В **самостоятельных работах** к этому уроку:

- Составление двумерных массивов (массивов, объектов) на основе табличных данных.
- Методы добавления элементов в многомерные массивы.
- Вывод элементов массивов.
- Работа со связанными массивами.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:



- **Практические** работы

